Step 1: Project setup + Supabase client + Google login

0. Create the app and install Supabase

```
npx create-next-app@latest my-shop    # choose: JavaScript, App Router, Tailwind = yes
cd my-shop
npm install @supabase/supabase-js
```

1. `.env.local` (project root, never commit this file)

```
# Supabase -> Project Settings -> API
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
```

`NEXT_PUBLIC_` means the browser can read it. The anon key is safe there
because Row Level Security (from schema.sql) protects the data.

2. `lib/supabase.js`: one shared Supabase client

```
import { createClient } from
"@supabase/supabase-js";

// Create ONE client and reuse it
everywhere in the app.
// It reads the URL and anon key from
.env.local.
export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
);
```

3. `components/AuthButton.jsx`: login / logout button + session tracking

```
"use client"; // runs in the browser
(needed for useState/useEffect)

import { useEffect, useState } from
"react";
import { supabase } from "@/lib/supabase";

export default function AuthButton() {
  // Holds the logged-in user, or null if
logged out
  const [user, setUser] = useState(null);
```

```
  useEffect(() => {
    // 1. On first load, check whether a
session already exists
    supabase.auth.getSession().then(({ data
}) => {
      setUser(data.session?.user ?? null);
    });

    // 2. Listen for login/logout events
and update the UI instantly
    const { data: listener } =
supabase.auth.onAuthStateChange(
      (_event, session) =>
setUser(session?.user ?? null)
    );

    // 3. Clean up the listener when the
component unmounts
    return () =>
listener.subscription.unsubscribe();
  }, []);

  // Sends the user to Google, then back to
our site
  async function login() {
    await supabase.auth.signInWithOAuth({
      provider: "google",
      options: { redirectTo:
window.location.origin }, // return to the
same site (localhost or Vercel)
    });
  }

  async function logout() {
```

```
    await supabase.auth.signOut();
  }

  // Logged in: show email + logout. Logged
out: show the Google button.
  return user ? (
    <div className="flex items-center gap-
3">
      <span className="text-sm">
{user.email}</span>
      <button onClick={logout}
className="rounded border px-3 py-1">
        Log out
      </button>
    </div>
  ) : (
    <button
      onClick={login}
      className="rounded bg-black px-4 py-2
text-white"
    >
      Continue with Google
    </button>
  );
}
```

4. Use it in the header (`app/layout.js`)

```
import "./globals.css";
import AuthButton from
"@/components/AuthButton";
```

```
export const metadata = { title: "My Shop"
};

export default function RootLayout({
children }) {
  return (
    <html lang="en">
      <body>
        <header className="flex items-
center justify-between border-b p-4">
          <a href="/" className="text-xl
font-bold">My Shop</a>
          <AuthButton />
        </header>
        <main className="mx-auto max-w-5xl
p-4">{children}</main>
      </body>
    </html>
  );
}
```

What each part does

- `createClient(...)`: connects your app to your Supabase project using the URL and anon key.
- `getSession()`: on page load, asks Supabase whether the user is already logged in.
- `onAuthStateChange(...)`: fires after Google redirects back, so the button flips to "Log out" without a refresh.

- `signInWithOAuth({ provider: "google" })`: starts the Google flow. Google sends the user to Supabase's callback URL, Supabase creates the session, then redirects to `redirectTo`.
- `redirectTo`: must be listed in Supabase -> Auth -> URL Configuration -> Redirect URLs (both `http://localhost:3000` and your Vercel URL), or login bounces to the wrong place.

Checklist before moving on

1. Ran `schema.sql` in Supabase SQL Editor.
2. Google provider enabled in Supabase with your client ID and secret.
3. Redirect URI in Google Cloud: `https://<project-ref>.supabase.co/auth/v1/callback`.
4. Your email added as a test user on the OAuth consent screen.
5. `npm run dev`, click "Continue with Google", confirm your email appears.